

Operating Systems

Inter-Process Communication (IPC)

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Source materials: this presentation includes topics discussed in Chapter 3 of the textbook, and it is an adaptation of the original book slides ch3.pptx.

Goals:

- Identify the components of a process
- Describe how processes are created and terminated
- Introduce design choices for interprocess communication

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes
 - Synchronization
 - Information sharing
- Cooperating processes need **interprocess communication (IPC)**
- IPC models
 - **Signaling** (does not allow data exchange)
 - **Shared memory**
 - **Message passing**

IPC for signaling

- A **signal** is an **asynchronous** software notification delivered by the kernel or another process to a process
- Signals are identified by a number and a symbolic name (for example, SIGINT number 2, SIGTERM number 15)
- They are primarily used for **events**, not data transfer
- Typical causes:
 - User actions (for example, Ctrl+C in a terminal)
 - Process actions (for example, `kill(pid, SIGTERM)`)
 - Kernel-detected conditions (for example, division by zero, invalid memory access)

- To see the signal numbers and names on a Linux system, use the `kill -l` command. This is the output on a recent Linux distribution. Not all signals are standard across Unix-like systems, but many are common.

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7
58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX			

Name	Typical source	Meaning
SIGINT	terminal Ctrl+C	interrupt process
SIGILL	CPU/kernel	illegal instruction
SIGUSR{1,2}	user-defined	application-specific
SIGTERM	kill default	polite termination request
SIGKILL	kill -9	immediate termination (cannot catch)
SIGCHLD	kernel	child process changed state (e.g. terminated)
SIGSTOP	kill -STOP	stop/suspend (cannot catch)
SIGTSTP	terminal Ctrl+Z	stop/suspend from terminal
SIGCONT	fg, bg, kill -CONT	continue stopped process
SIGSEGV	kernel/MMU	invalid memory access
SIGFPE	CPU/kernel	arithmetic exception
SIGPIPE	write on closed pipe/socket	broken pipe
SIGCHLD	kernel	child process changed state
SIGHUP	terminal/session end	hangup/reload convention

Why Signals Are Useful

- **Process control**: terminate, stop, continue processes
- **Service management**: graceful shutdown and reload (SIGTERM/SIGHUP)
- **Child management**: parent notified with SIGCHLD
- **Timeouts and timers**: SIGALRM, POSIX timers
- **Fault reporting**: SIGSEGV, SIGBUS, SIGILL
- **Low-overhead notifications**: trigger actions without the overhead of more complex IPC mechanisms

- A signal is **generated**
- It becomes **pending** for the target process or thread
- If not blocked, it is **delivered**
- On delivery, one of three dispositions is applied:
 - **Default action** - the kernel performs the predefined action for that signal. Usually, this is to terminate the process
 - **Ignore** - the signal is discarded and has no effect on the process
 - **User-defined handler** - the process has registered a custom function to execute when the signal is delivered. After handler returns, normal execution resumes

- From shell:

```
kill -TERM <PID> # default signal is TERM
kill -KILL <PID> # force kill
kill -STOP <PID> # suspend process
kill -CONT <PID> # resume process
pkill <PROCESS_NAME> # signal by process name
```

- From C code:

```
kill(pid, SIGTERM);
raise(SIGINT); // send to current process
```

Installing a Handler with `signal()` (Legacy)

```
#include <signal.h>

void on_int(int sig) {
    /* do something here */
}

int main(void) {
    signal(SIGINT, on_int);
    while (1) { }
}
```

- Simple API, available in older codebases
- Historically inconsistent semantics across systems
- In modern POSIX/Linux code, prefer `sigaction()`

Line-by-Line: `signal()` Example

- `#include <signal.h>`: imports declarations for signals and `signal()`
- `void on_int(int sig) { ... }`: defines the handler for `SIGINT`
- Comment inside handler: reminder to keep logic minimal and async-signal-safe
- `int main(void)`: program entry point
- `signal(SIGINT, on_int);`: installs `on_int` as disposition for `SIGINT`
- `while (1) { }`: infinite loop keeps the process alive, waiting for a signal
- Runtime behavior: pressing `Ctrl+C` sends `SIGINT`, then the handler runs

Installing a Handler with `sigaction()` (Preferred)

```
#include <signal.h>

void on_int(int sig) {
    /* do something here */
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_int;
    sigemptyset(&sa.sa_mask); // extra signals blocked in handler
    sa.sa_flags = SA_RESTART; // restart some interrupted syscalls
    sigaction(SIGINT, &sa, NULL);
    while (1) { }
}
```

Line-by-Line: sigaction() Example

- `#include <signal.h>`: provides struct `sigaction` and related APIs
- `void on_int(int sig) { ... }`: handler function called on `SIGINT`
- `struct sigaction sa = {0};`: zero-initialize all fields for predictable setup
- `sa.sa_handler = on_int;`: set the simple one-argument handler
- `sigemptyset(&sa.sa_mask);`: no extra signals blocked while handler runs
- `sa.sa_flags = SA_RESTART;`: automatically restart some interrupted system calls
- `sigaction(SIGINT, &sa, NULL);`: install this action for `SIGINT`
- `while (1) { }`: keep the process running so signals can be received

signal() **vs** sigaction()

- `signal()`:
 - Smaller API, but legacy and less explicit
 - Behavior may vary (especially in older Unix variants)
- `sigaction()`:
 - POSIX-defined, precise and portable behavior
 - Controls flags (for example, `SA_RESTART`, `SA_SIGINFO`)
 - Controls temporary blocking during handler via `sa_mask`
 - Can inspect previous action (third argument)
- Practical rule: use `sigaction()` in production code

- A **signal mask** is the set of currently blocked signals for a process/thread
- If a blocked signal arrives, it becomes **pending** and delivery is deferred
- When unblocked, pending signals can be delivered
- Standard signals are usually coalesced (multiple arrivals may become one pending signal)
- `sigprocmask()` operations:
 - `SIG_BLOCK`: add signals to mask
 - `SIG_UNBLOCK`: remove signals from mask
 - `SIG_SETMASK`: replace current mask
- `SIGKILL` and `SIGSTOP` cannot be blocked

Blocking Signals with `sigprocmask()`

```
#include <signal.h>

int main(void) {
    sigset_t set, oldset;
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGTERM);

    /* Block SIGINT and SIGTERM */
    sigprocmask(SIG_BLOCK, &set, &oldset);

    /* critical section: protected from these signals */

    /* Restore previous mask */
    sigprocmask(SIG_SETMASK, &oldset, NULL);
}
```

Line-by-Line: sigprocmask() Example

- `#include <signal.h>`: declares signal set types and `sigprocmask()`
- `sigset_t set, oldset`;: `set` is the new mask fragment, `oldset` stores previous mask
- `sigemptyset(&set)`;: start from an empty signal set
- `sigaddset(&set, SIGINT)`; and `sigaddset(&set, SIGTERM)`;: add signals to be blocked
- `sigprocmask(SIG_BLOCK, &set, &oldset)`;: block those signals and save old mask atomically
- Critical section: code executes without delivery of `SIGINT/SIGTERM`
- `sigprocmask(SIG_SETMASK, &oldset, NULL)`;: restore original mask after critical work
- Effect: signals arriving while blocked become pending and may be delivered after unblocking

- When a signal handler is running, other instances of the same signal are automatically blocked, become pending, and are unblocked after the handler returns
- Different signals are not automatically blocked, so they can interrupt the handler (for example, `SIGINT` can interrupt a `SIGTERM` handler if not blocked)
- The `sa_mask` field of `struct sigaction` allows you to specify additional signals to block while the handler runs. They become pending and will be delivered after the handler returns
- **Note:** multiple identical signals that arrive while blocked will be coalesced into a single pending signal (not true for real-time signals, which are queued)

Raising a Signal with `raise()`

- `raise(sig)` sends signal `sig` to the **calling process** itself
- It is equivalent in spirit to `kill(getpid(), sig)` for process-directed signals
- Typical uses:
 - Trigger your own handler for testing/debugging
 - Force a controlled path (for example, graceful stop on `SIGTERM`)
- Return value:
 - 0 on success
 - non-zero on failure (and `errno` is set)

Example: Self-Signal with `raise(SIGUSR1)`

```
#include <signal.h>
#include <stdio.h>

static void on_usr1(int sig) {
    // handle SIGUSR1
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGUSR1, &sa, NULL);

    raise(SIGUSR1); // send SIGUSR1 to self

    return 0;
}
```

The pause() System Call

- pause() suspends the calling process until a signal is delivered that either terminates the process or invokes a handler
- Commonly used to wait for signals in simple event-driven programs
- Typical usage: main loop waits for signals, handler sets a flag or performs minimal work

```
#include <signal.h>
#include <unistd.h>

void on_usr1(int sig) {
    // handle SIGUSR1
}

int main(void) {
    signal(SIGUSR1, on_usr1); //signal instead of sigaction for brevity
    while (1) {
        pause(); // sleep until a signal arrives
    }
}
```

Raising a Signal with `alarm()`

- `alarm(seconds)` schedules delivery of `SIGALRM` to the calling process after `seconds`
- Useful for implementing simple timeouts
- Key behavior:
 - Only one alarm timer per process
 - A new `alarm()` call replaces the previous one
 - `alarm(0)` cancels any pending alarm
- Return value: remaining seconds from any previously scheduled alarm

Example: Timeout with alarm(3)

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void on_alarm(int sig) {
    // handle SIGALRM
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_alarm;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGALRM, &sa, NULL);

    alarm(3); // raise SIGALRM after 3 seconds
    pause(); // sleep until a signal arrives
    puts("Timeout reached");
}
```


Signal Handler Rules. Important!

- Handlers run asynchronously: they can interrupt normal control flow at any point
- Only call **async-signal-safe** functions inside handlers
 - Safe example: `write`
 - Unsafe examples: `printf`, `malloc`, most library functions
 - [man signal-safety](#)
 - ... I know, some example do use `printf` in handlers, but this is not recommended for production code!
- Keep handler logic minimal:
 - Set a flag
 - Let the main loop perform complex operations and I/O based on that flag

Example: Graceful Shutdown Pattern

```
#include <signal.h>
#include <stdatomic.h>

static volatile sig_atomic_t stop = 0;

static void on_term(int sig) {
    stop = 1;
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_term;
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    while (!stop) {
        /* do useful work */
    }
    /* close files, flush state, release resources */
}
```

- If a parent installs a signal handler and then calls `fork()`, the child inherits the same signal dispositions
- This means the child will run the same user-defined handler when that signal is delivered
- The child also inherits the current signal mask (blocked/unblocked set)
- Pending signals are **not** inherited by the child
- Parent and child are separate processes: a signal sent to one does not automatically go to the other
- After `exec()`, signal handlers are reset to default, while ignored signals remain ignored

Practical note: after `fork()`, many programs reconfigure handlers in child to define distinct behavior.

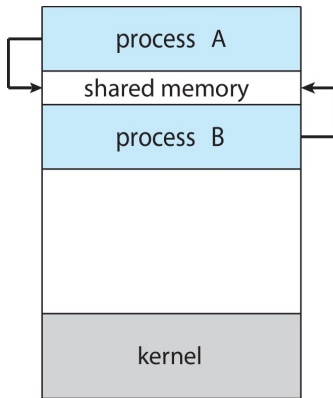
- Range usually from SIGRTMIN to SIGRTMAX
- Differ from standard signals:
 - They are queued (not collapsed)
 - Delivered in priority/order rules defined by POSIX/Linux
 - Can carry payloads using `sigqueue`
- Useful for event notifications where no signal occurrence should be lost
- **We will not cover real time OS and real time signals in this course!**

- Signals are the classic Unix/Linux primitive for asynchronous notifications
- Understand default actions before deploying services
- Use `sigaction` instead of legacy `signal()`
- Write minimal, safe handlers and perform heavy work in normal code paths
- Remember: `SIGKILL` and `SIGSTOP` are always enforced by the kernel

IPC for data exchange

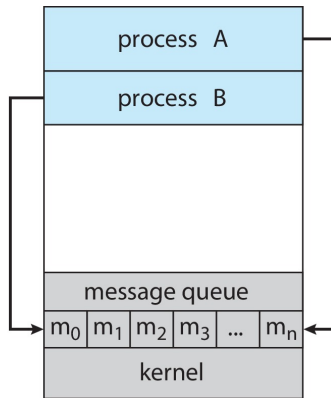
- Paradigm for cooperating processes:
 - **producer** process produces information that is consumed by a **consumer** process
- Two variations:
 - **unbounded-buffer**: no practical limit on the size of the buffer. Producer never waits. Consumer waits if there is no buffer to consume
 - **bounded-buffer** assumes that there is a fixed buffer size. Producer must wait if all buffers are full. Consumer waits if there is no buffer to consume

(a) Shared memory



(a)

(b) Message passing.



(b)

Shared memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes, not the operating system.
- Need for a **synchronization mechanism** to coordinate access to the shared memory.

- Shared data

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0; // points to the next free position in the buffer
int out = 0; // points to the next full position in the buffer
```

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* buffer full - do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* buffer empty - do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in next consumed */
}
```

- `mmap()` can be used to create an anonymous shared memory region that is shared between related processes (for example, parent and child after `fork()`)

```
int fd = -1; // not backed by a file
size_t size = 4096; // size of shared memory region
void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, fd,
0);
```

- `mmap()` parameters:
 - `NULL`: let the kernel choose the address
 - `size`: size of the shared memory region
 - `PROT_READ | PROT_WRITE`: allow read/write access
 - `MAP_SHARED | MAP_ANONYMOUS`: shared mapping not backed by a file
 - `fd` and `offset (0)` are ignored for anonymous mappings
- The returned pointer `ptr` can be used for reading/writing shared data between the processes that inherit this mapping

- Process first creates shared memory segment
`shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`
- Also used to open an existing shared memory named `name` already created by another process
- Set the size of the object
`ftruncate(shm_fd, 4096);`
- Use `mmap()` to memory-map a file pointer to the shared memory object
- Reading and writing to shared memory is done by using the pointer returned by `mmap()`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main(){
    /* the size (in bytes) of shm obj */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";
    /* shared memory file descriptor */
    int fd;
    /* pointer to shared memory object */
    char *ptr;
```

```
/* create the shared memory object */
fd = shm_open(name, O_CREAT | O_RDWR,
              0666);
/* configure the size of the shared
   memory object */
ftruncate(fd, SIZE);
/* memory map the shared memory object
   */
ptr = (char *)
mmap(0, SIZE, PROT_READ | PROT_WRITE,
     MAP_SHARED, fd, 0);
/* write to the shared memory object
   */
sprintf(ptr,"%s", message_0);
ptr += strlen(message_0);
sprintf(ptr,"%s", message_1);
ptr += strlen(message_1);
return 0;
```

```
}
```



```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main(){
    /* the size (in bytes) of shared
       memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int fd;
    /* pointer to shared memory object */
    char *ptr;

    /* open the shared memory object */
    fd = shm_open(name, O_RDONLY, 0666);
    /* memory map the shared memory object
       */
    ptr = (char *)
    mmap(0, SIZE, PROT_READ, MAP_SHARED,
        fd, 0);
    /* read from the shared memory object
       */
    printf("%s", (char *)ptr);
    /* remove the shared memory object */
    shm_unlink(name);
    return 0;
}
```

Message passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - `send(message)`
 - `receive(message)`
- The message size is either fixed or variable

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bi-directional?

- **Physical Implementation**

- **Shared memory:** processes communicate by reading/writing in a common memory space
- **Filesystem:** processes exchange data by reading/writing to files
- **Hardware bus:** communication through a shared physical channel within the same machine
- **Network:** data exchange over local or wide-area networks

- **Logical Implementation**

- **Direct vs. indirect:** processes communicate directly or via intermediaries (e.g., mailboxes)
- **Synchronous vs. asynchronous:** sender waits for receiver or continues independently
- **Automatic vs. explicit buffering:** system manages message storage or programmer controls it

- Processes must name each other explicitly
 - `send (P, message)` – send a message to process P
 - `receive(Q, &message)` – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

- Create a new mailbox (port)
 - Send and receive messages through mailbox
 - Delete a mailbox
- Primitives are defined as
 - `send(A, message)` – send a message to mailbox A
 - `receive(A, &message)` – receive a message from mailbox A

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A
 - P_1 sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
 - **Blocking send**: the sender is blocked until the message is received
 - **Blocking receive**: the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send**: the sender sends the message and continue
 - **Non-blocking receive**: the receiver receives
 - A valid message or
 - Null message
- Different combinations possible
 - If both send and receive are blocking, we have a **rendezvous**

Producer

```
message next_produced;  
while (true) {  
    /* produce an item in next_produced */  
    send(next_produced);  
}
```

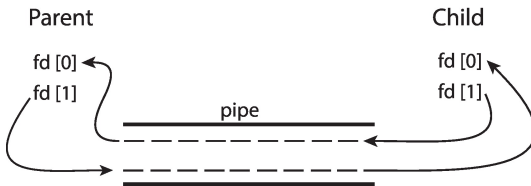
Consumer

```
message next_consumed;  
while (true) {  
    receive(&next_consumed);  
    /* consume the item in next_consumed */  
}
```

- Queue of messages attached to the link.
- Implemented in one of three ways
 - **Zero capacity**: no messages are queued on a link. Sender must wait for receiver (rendezvous)
 - **Bounded capacity**: finite length of n messages. Sender must wait if link full
 - **Unbounded capacity**: infinite length. Sender never waits

- Acts as a conduit allowing two processes to communicate
- Issues
 - Is communication unidirectional or bidirectional?
 - In the case of two-way communication, is it half or full-duplex?
 - Must there exist a relationship (i.e., parent-child) between the communicating processes?
 - Can the pipes be used over a network?
- **Ordinary pipes**: cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes**: can be accessed without a parent-child relationship.

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore **unidirectional**
- Require parent-child relationship between communicating processes



- Windows calls these anonymous pipes

The `pipe()` system call

```
int pipe(int fd[2]);
```

- Takes an array of two integers as an argument, which will be filled with the file descriptors for the **read** and **write** ends of the pipe
- Returns 0 on success and -1 on failure (with `errno` set)
- The pipe is unidirectional: data written to `fd[1]` can be read from `fd[0]`
- The pipe has a maximum size (buffer capacity), and writes will block if the buffer is full, while reads will block if the buffer is empty
- Reading from the read end of the pipe will return 0 (EOF) if the write end is closed and all data has been read
- SIGPIPE signal is sent to the writing process if it attempts to write to a pipe with no readers. If this signal is not handled, the process will be terminated. To avoid this, the process can ignore SIGPIPE or handle it appropriately.
- If SIGPIPE is handled or ignored, writing to the write end of the pipe will return -1 with EPIPE if the read end is closed

Pipe example

```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#define BUF_SIZE 25
#define READ_END 0
#define WRITE_END 1
int main(void) {
    char wmsg[BUF_SIZE] = "Greetings";
    char rmsg[BUF_SIZE];
    int fd[2];
    pid_t pid;
    /* create the pipe + fork*/
    if (pipe(fd) == -1) {
        perror("pipe"); return 1;}
    pid = fork();
    if (pid < 0) { /* error occurred */
        perror("fork"); return 1;}
```

```
    if (pid > 0) { /* parent process */
        /* close the unused end of the pipe */
        close(fd[READ_END]);
        /* write to the pipe */
        write(fd[WRITE_END], wmsg, strlen(wmsg)
            +1);
        /* close the write end of the pipe */
        close(fd[WRITE_END]);
    } else { /* child process */
        /* close the unused end of the pipe */
        close(fd[WRITE_END]);
        /* read from the pipe */
        read(fd[READ_END], rmsg, BUF_SIZE);
        printf("read %s",rmsg);
        /* close the read end of the pipe */
        close(fd[READ_END]);
    }
    return 0;
}
```


- Ordinary pipes are unidirectional and require a parent-child relationship between communicating processes
- Each pipe has a read end and a write end, represented by file descriptors returned by `pipe()`
- Each process **must** close the unused end of the pipe to avoid resource leaks and ensure proper behavior

- Through the use of multiple pipes, more complex communication patterns can be established between processes
- For example, two pipes can be used to achieve bidirectional communication between a parent and child process
- Multiple pipes allow for communication between multiple processes, such as a parent communicating with several child processes, or even a more complex network of processes communicating with each other
- See example [05-pipe-ring.c](#) for a demonstration of a ring communication pattern using pipes
- See example [06-pipe-star.c](#) for a demonstration of a star communication pattern using pipes

- Named pipes can be used for communication between **unrelated** processes
- They can be accessed through a name in the filesystem (that usually is not a real file in the filesystem, but a special file in memory)
- They are created using the `mkfifo()` system call

```
mkfifo("myfifo", 0666);
```

- "myfifo" is the name of the FIFO special file that will be created in the filesystem. The default location is the current working directory, but an absolute path can also be used
- 0666 is the permission mode for the FIFO special file, expressed in octal.

- Once a named pipe is created, processes can open it for reading or writing using standard file I/O system calls (e.g., `open()`, `read()`, `write()`)
- The named pipe behaves like a regular file, but it is actually a special file that provides a communication channel between processes
- Multiple processes can open the same named pipe for reading and writing, allowing for more complex communication patterns

- When a process is done using a named pipe, it should close the file descriptor associated with it using `close()`
- If a process is the last one to close the named pipe, the special file will still exist in the filesystem until it is explicitly removed using `unlink()`

`unlink("myfifo");`

- This will delete the named pipe "myfifo" from the filesystem. It is important to clean up named pipes when they are no longer needed to avoid cluttering the filesystem with unused special files

Unix Sockets

- Sockets are more powerful than ordinary pipes
- Communication is **bidirectional**
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the socket for communication
- Can be easily generalized to network communications

- Unix domain sockets (also known as IPC sockets) are a type of socket that allows for communication between processes on the same host

```
socket(AF_UNIX, SOCK_STREAM, 0);
```

- AF_UNIX specifies the address family for the socket
- SOCK_STREAM indicates that the socket will provide a reliable, connection-oriented byte stream service
- The third argument (protocol) is typically set to 0 to select the default protocol for the given address family and socket type
- the return value is a file descriptor that can be used for subsequent socket operations (e.g., bind, listen, accept, connect, read, write)

- A socket server typically performs the following steps:
 - Create a socket using `socket()`
 - Bind the socket to an address (e.g., a file path for Unix domain sockets) using `bind()`
 - Listen for incoming connections using `listen()`
 - Accept a client connection using `accept()`
 - Read from and write to the client socket using `read()` and `write()`
 - Clean up by closing the client and server sockets and unlinking any resources if necessary

- A socket client typically performs the following steps:
 - Create a socket using `socket()`
 - Connect to the server socket using `connect()`
 - Read from and write to the server socket using `read()` and `write()`
 - Clean up by closing the client socket

Sockets can implement various communication protocols, including:

- **Unix domain sockets:** for communication between processes on the same host
- **Internet domain sockets:** for communication over a network using protocols like TCP/IP
- **Raw sockets:** for low-level network communication, allowing direct access to network protocols

Lots of variations and options available for socket programming, including different socket types (e.g., `SOCK_STREAM`, `SOCK_DGRAM`) and various socket options for configuring behavior

We will only use Unix domain sockets. There is a lot of boilerplate code involved in socket programming, and we will not cover all the details. The examples provided are meant to illustrate the basic concepts and usage of sockets for interprocess communication on the same host. For more complex socket programming (e.g., network sockets, raw sockets), additional resources and documentation should be consulted.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#define SOCKET_PATH "/tmp/sock"
#define BUF_SIZE 25
int main(void) {
    int srv_fd, cli_fd;
    struct sockaddr_un srv_addr, cli_addr;
    socklen_t client_addr_len;
    char wmsg[BUF_SIZE] = "Greetings";
    /* Create a Unix domain socket */
    srv_fd=socket(AF_UNIX, SOCK_STREAM, 0);
    if (srv_fd==-1) {
        perror("socket"); return 1;}
    unlink(SOCKET_PATH);
    memset(&srv_addr, 0, sizeof(struct
        sockaddr_un));
```

```
    srv_addr.sun_family=AF_UNIX;
    strncpy(srv_addr.sun_path, SOCKET_PATH,
        sizeof(srv_addr.sun_path) - 1);
    /* Bind the socket */
    if (bind(srv_fd, (struct sockaddr *)&
        srv_addr, sizeof(struct sockaddr_un)
        ) == -1) {
        perror("bind"); return 1;}
    /* Listen for incoming connections */
    if (listen(srv_fd, 5) == -1) {
        perror("listen"); return 1;}
    /* Accept a client connection */
    client_addr_len = sizeof(struct
        sockaddr_un);
    cli_fd = accept(srv_fd, (struct sockaddr
        *)&cli_addr, &client_addr_len);
    if (cli_fd == -1) {
        perror("accept"); return 1;}
```

```
/* Write the message to the client socket */
if (write(cli_fd, wmsg, strlen(wmsg) + 1) == -1) {
    perror("write"); return 1;}
printf("Message sent: '%s'\n", wmsg);
/* Clean up */
close(cli_fd);
close(srv_fd);
unlink(SOCKET_PATH);
return 0;
}
```

- `socket()`: creates a new socket and returns a file descriptor for it
- `bind()`: assigns a local address (e.g., file path) to the socket
- `listen()`: marks the socket as a passive socket that will be used to accept incoming connection requests
- `accept()`: blocks until an incoming connection request is received, then returns a new file descriptor for the accepted connection

- The `bind()` system call is used to associate a socket with a specific local address (e.g., file path for Unix domain sockets)
- For Unix domain sockets, the address is specified using a `sockaddr_un` structure, which includes the address family (`AF_UNIX`) and the file path where the socket will be created
- The file path specified in `sun_path` must not already exist, as `bind()` will fail if the file already exists. It is common practice to call `unlink()` on the socket path before calling `bind()` to ensure that any existing socket file is removed

- The `listen()` system call is used to mark a socket as a passive socket that will be used to accept incoming connection requests
- The first argument to `listen()` is the file descriptor of the socket that has been bound to a local address using `bind()`
- The second argument to `listen()` specifies the maximum number of pending connections that can be queued for acceptance. If the queue is full and additional connection requests arrive, they may be refused or ignored by the system
- After calling `listen()`, the server socket is ready to accept incoming connections using the `accept()` system call

- The `accept()` system call is used to accept an incoming connection request on a socket that has been marked as a passive socket using `listen()`
- The first argument to `accept()` is the file descriptor of the socket that has been marked as passive
- The second argument is a pointer to a `sockaddr` structure where the address of the connecting entity will be stored
- The third argument is a pointer to a `socklen_t` variable that initially contains the size of the `sockaddr` structure and will be updated to the actual size of the address
- `accept()` returns a new file descriptor for the accepted connection, which can be used for communication with the client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#define SOCKET_PATH "/tmp/sock"
#define BUF_SIZE 25
int main(void) {
    int cli_fd;
    struct sockaddr_un srv_addr;
    char rmsg[BUF_SIZE];
    ssize_t bytes_read;
    /* Create a unix socket */
    cli_fd=socket(AF_UNIX, SOCK_STREAM, 0);
    if (cli_fd==-1) {
        perror("socket"); return 1;}
    memset(&srv_addr, 0, sizeof(struct
        sockaddr_un));
```

```
server_addr.sun_family = AF_UNIX;
strncpy(server_addr.sun_path,
        SOCKET_PATH, sizeof(server_addr.
        sun_path) - 1);
/* Connect to the server socket */
if (connect(cli_fd, (struct sockaddr *)&
        server_addr, sizeof(struct
        sockaddr_un)) == -1) {
    perror("connect"); return 1;}
/* Read the message from the socket */
bytes_read = read(cli_fd, rmsg, sizeof(
        rmsg) - 1);
if (bytes_read == -1) {
    perror("read"); return 1;}
rmsg[bytes_read] = '\0';
printf("read: \"%s\"\n", rmsg);
close(cli_fd);
return 0;
}
```

- `socket()`: creates a new socket and returns a file descriptor for it
- `connect()`: initiates a connection to the server socket specified by the address

- The `connect()` system call is used by a client process to establish a connection to a server socket
- The first argument to `connect()` is the file descriptor of the socket that the client has created using `socket()`
- The second argument is a pointer to a `sockaddr` structure that specifies the address of the server socket to which the client wants to connect
- The third argument is the size of the `sockaddr` structure
- If the connection is successful, `connect()` returns 0. If the connection fails, it returns `-1` and sets `errno` to indicate the error